Iterator Pattern

Architecture of Computer Games
SE 456

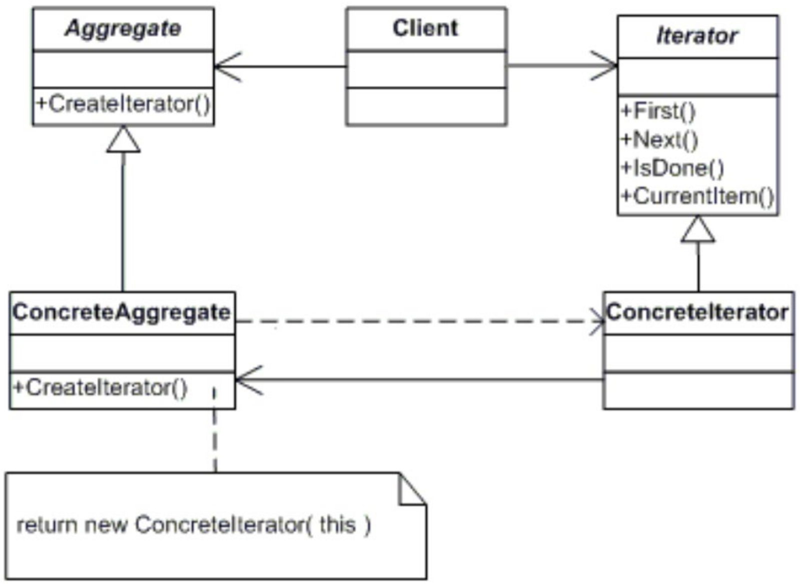
Ed Keenan

12 January 2022

13.0.9.3.9 Mayan Long Count

Iterator

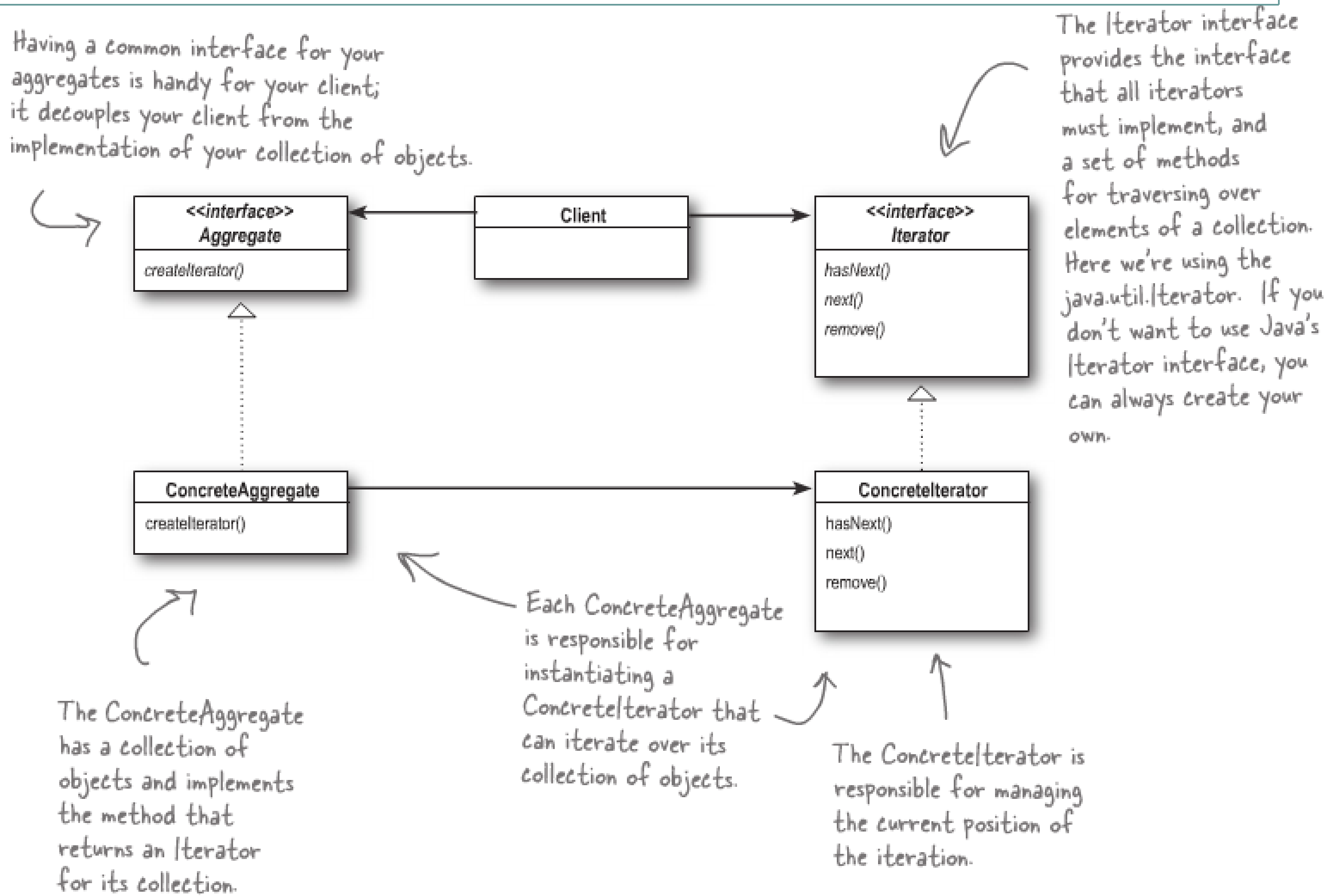
- Intent
  - Provide a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
  - Provides an abstraction that makes it possible to decouple collection classes and algorithms.
- Remarks
  - Promote to "full object status" the traversal of a collection.
  - Polymorphic traversal



Iterator

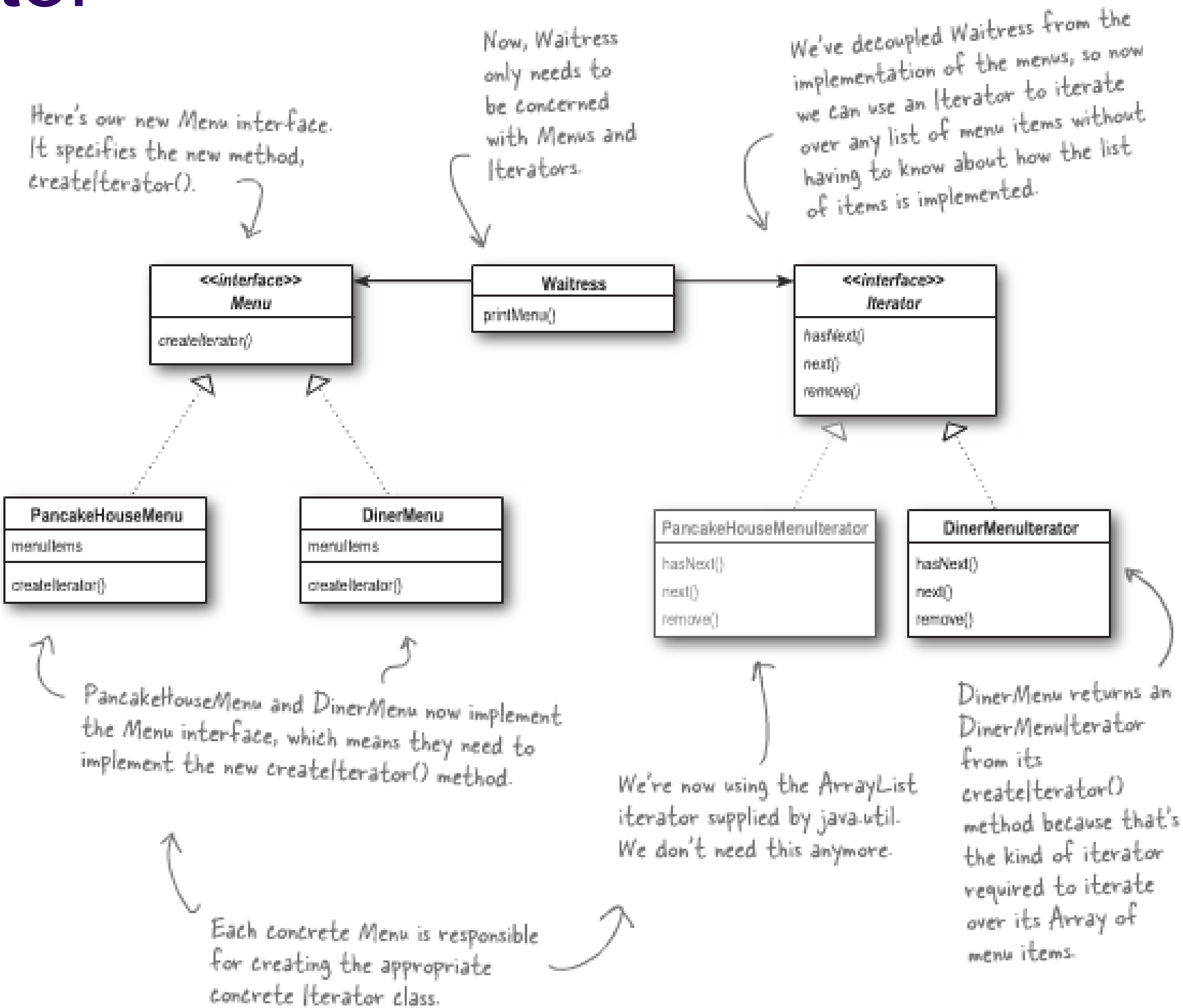
- Iterator Pattern provides a way to access the elements of an aggregate object sequentially without exposing its underlying representation.
- It also places the task of traversal on the iterator object, not on the aggregate, which simplifies the aggregate interface and implementation, and places the responsibility where it should be.

Iterator



Having a common interface for your aggregates is handy for your client; it decouples your client from the implementation of your collection of objects.
<<interface>>
Aggregate
createIterator()
Client
<<interface>>
Iterator
hasNext()
next()
remove()
The Iterator interface provides the interface that all iterators must implement, and a set of methods for traversing over elements of a collection. Here we're using the java.util.Iterator. If you don't want to use Java's Iterator interface, you can always create your own.
ConcreteAggregate
createIterator()
ConcreteIterator
hasNext()
next()
remove()
Each ConcreteAggregate is responsible for instantiating a ConcreteIterator that can iterate over its collection of objects.
The ConcreteAggregate has a collection of objects and implements the method that returns an Iterator for its collection.
The ConcreteIterator is responsible for managing the current position of the iteration.

Iterator



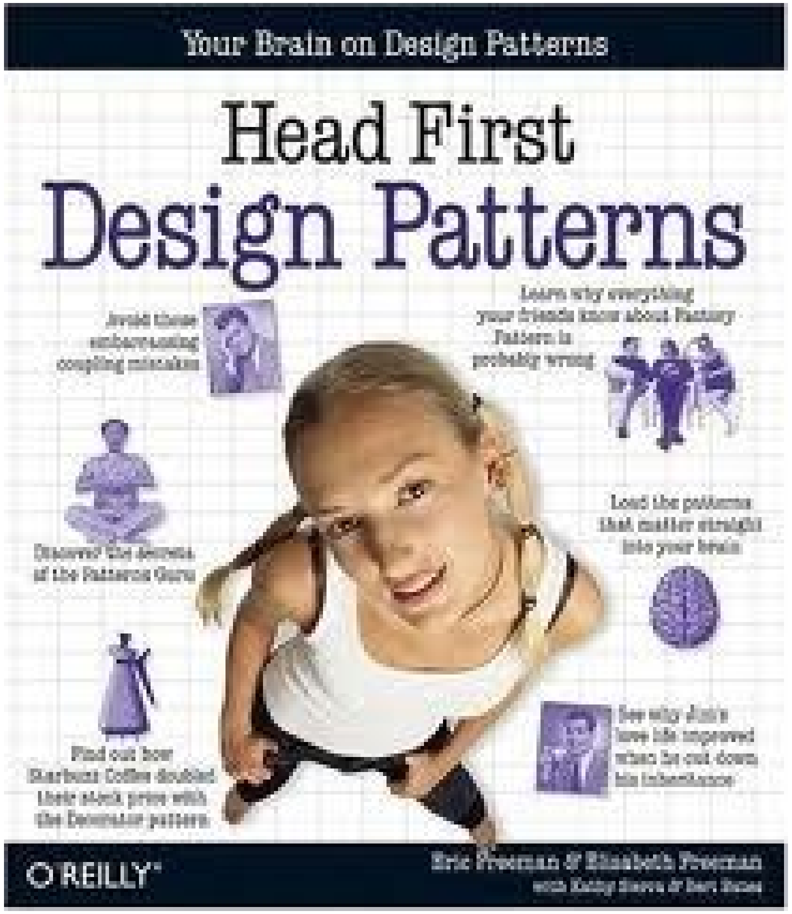
Now, Waitress only needs to be concerned with Menus and Iterators.
We've decoupled Waitress from the implementation of the menus, so now we can use an Iterator to iterate over any list of menu items without having to know about how the list of items is implemented.
Here's our new Menu interface. It specifies the new method, createIterator().
<<interface>>
Menu
createIterator()
Waitress
printMenu()
<<interface>>
Iterator
hasNext()
next()
remove()
PancakeHouseMenu
menuItems
createIterator()
DinerMenu
menuItems
createIterator()
PancakeHouseMenuIterator
hasNext()
next()
remove()
DinerMenuIterator
hasNext()
next()
remove()
PancakeHouseMenu and DinerMenu now implement the Menu interface, which means they need to implement the new createIterator() method.
We're now using the ArrayList iterator supplied by java.util. We don't need this anymore.
DinerMenu returns an DinerMenuIterator from its createIterator() method because that's the kind of iterator required to iterate over its Array of menu items.
Each concrete Menu is responsible for creating the appropriate concrete Iterator class.

Iterators in Game Engine

- Doing actions on a collection of objects?
  - List of objects need to perform a similar actions
    - Anytime you have a need for STL containers
  - Keep concept
    - You can add / remove
    - Walk through the list
      - Without knowing or caring the length
  - Just a few
    - Collision objects
    - Graphical objects
    - Cameras

Design Patterns

- Easy tutorial book on design patterns
  - Get your copy today
  - Ecopy at Depaul Library through the safari link



Thank You!

- Questions?